

Week 4 — Advanced SQL & Transaction Management

CST4714 Database
Administration

Functions, views, triggers,
constraints, and ACID guarantees

Agenda

Review advanced SQL building blocks and server-side programming

Deep dive on constraints, triggers, and enforcing business rules

Transaction processing: ACID properties and isolation levels

Hands-on lab preview: create views/triggers, test constraints, manage transactions

Kick off midterm project proposal planning

Advanced SQL Building Blocks

Functions: reusable logic executed on the server, authored in SQL or PL/pgSQL

Views: saved query abstractions that secure and simplify data access

Triggers: event-driven hooks invoking functions before/after DML

Constraints: declarative guards ensuring structural and business integrity

Transactions: atomic units of work that maintain database consistency

User-Defined Functions Overview

```
-- SQL vs PL/pgSQL
function creation
CREATE FUNCTION
util.add_tax(amount
numeric)
RETURNS numeric LANGUAGE
sql AS
$$ SELECT amount *
1.08875; $$;

CREATE OR REPLACE FUNCTION
audit.log_change()
RETURNS trigger LANGUAGE
plpgsql AS $$
BEGIN
    INSERT INTO
audit.log(table_name,
payload)
VALUES (TG_TABLE_NAME,
row_to_json(NEW));
    RETURN NEW;
END;
$$;
```

Languages: SQL for simple expressions, PL/pgSQL for procedural logic, extensions for Python/JavaScript

Volatility categories control planner assumptions (`IMMUTABLE`, `STABLE`, `VOLATILE`)

Parameter mode support (`IN`, `OUT`, `VARIADIC`) and default values

Security context: `SECURITY INVOKER` (default) vs `SECURITY DEFINER` for elevated tasks

Creating PL/pgSQL Functions

```
CREATE OR REPLACE
FUNCTION
finance.apply_discount
(
    order_id uuid,
    pct numeric
) RETURNS VOID
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE orders
        SET order_total =
order_total * (1 -
pct)
    WHERE id =
order_id;
    PERFORM
pg_notify('pricing',
order_id::text);
END;
$$;
```

Definition pattern:
`CREATE OR
REPLACE FUNCTION
schema.fn(args)
RETURNS type
LANGUAGE plpgsql
AS \$\$... \$\$`;

Use `DECLARE` block
for variables and
`BEGIN ... END` for
logic

Return scalars, sets
(`RETURNS TABLE`),
or `SETOF`
composite types

Add unit tests with
`DO \$\$` blocks or
extension
frameworks (pgTAP)
to validate
behaviour

Views & Materialized Views

```
CREATE VIEW  
reporting.active_customers  
AS  
SELECT id, full_name,  
email  
FROM customers  
WHERE status = 'active';
```

```
grant select ON  
reporting.active_customers  
TO analyst;
```

```
CREATE MATERIALIZED VIEW  
reporting.monthly_sales AS  
SELECT date_trunc('month',  
created_at) AS month,  
SUM(order_total) AS  
revenue  
FROM orders  
GROUP BY 1;
```

`CREATE VIEW` stores query definitions; security barrier ensures predicate pushdown safety

Use views to expose curated subsets, hide join complexity, and enforce row-level permissions

Materialized views persist results; require `REFRESH MATERIALIZED VIEW` to stay current

Consider indexes on materialized view columns for reporting workloads

Designing Robust Views

```
CREATE VIEW  
sales.premium_orders AS  
SELECT * FROM orders WHERE  
order_total > 500  
WITH CHECK OPTION;
```

```
ALTER VIEW  
sales.premium_orders SET  
(security_barrier = true);
```

Apply column aliases and document semantics to improve discoverability

Use `WITH CHECK OPTION` to prevent updates that violate view filters

Leverage `security\_barrier = on` for views used in security-sensitive contexts

Version control view definitions alongside migrations to track schema evolution

Constraint Toolbox

```
CREATE TABLE
reservations (
    id serial
PRIMARY KEY,
    room_id int NOT
NULL,
    starts_at
tstzrange NOT NULL,
    CONSTRAINT
no_overlap
        EXCLUDE USING
gist (room_id WITH
=, starts_at WITH
&&)
);

ALTER TABLE bookings
ADD CONSTRAINT
price_positive CHECK
(total >= 0);
```

Structural: `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `FOREIGN KEY` maintain relational integrity

Business logic: `CHECK` constraints evaluate expressions per row

Exclusion constraints (`USING gist`) prevent conflicting ranges or schedules

Domain constraints let you define reusable typed validations across tables

Advanced Constraint Management

```
ALTER TABLE orders
  ADD CONSTRAINT
  fk_customer
  FOREIGN KEY
  (customer_id)
  REFERENCES
  customers(id)
  ON DELETE
  RESTRICT;
```

```
ALTER TABLE invoices
  ADD CONSTRAINT
  invoices_unique
  UNIQUE
  (invoice_number)
  DEFERRABLE INITIALLY
  DEFERRED;
```

Deferrable constraints postpone enforcement until transaction commit (`DEFERRABLE INITIALLY DEFERRED`)

Use `ALTER TABLE ... VALIDATE CONSTRAINT` when backfilling large datasets

Document cascading behaviours for foreign keys (`ON DELETE CASCADE/SET NULL/RESTRICT`)

Prefer declarative constraints over trigger-based validation when possible for planner awareness

Trigger Fundamentals

```
CREATE OR REPLACE FUNCTION
audit.order_history()
RETURNS trigger LANGUAGE
plpgsql AS $$
BEGIN
    INSERT INTO
    audit.orders(order_id,
    payload, changed_at)
    VALUES (NEW.id,
    row_to_json(NEW), now());
    RETURN NEW;
END;
$$;

CREATE TRIGGER
orders_after_update
AFTER UPDATE ON orders
FOR EACH ROW EXECUTE
FUNCTION
audit.order_history();
```

Triggers fire on table events:
`INSERT`, `UPDATE`, `DELETE`,
`TRUNCATE`

Timing: `BEFORE` for
validation/mutation, `AFTER` for
logging or side effects

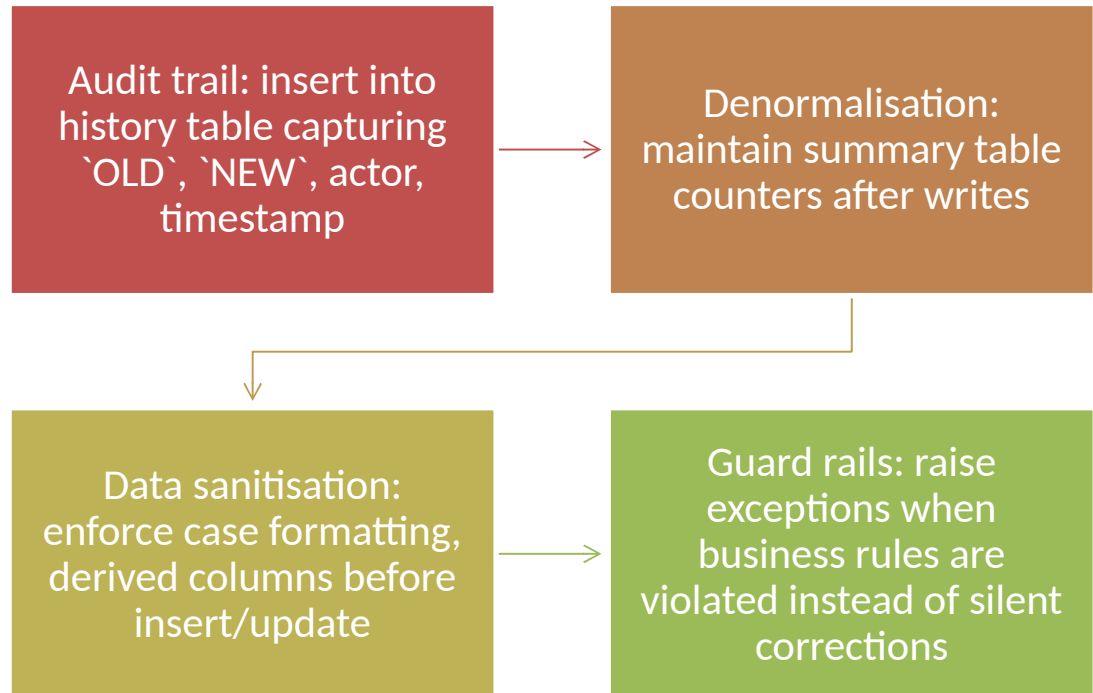
Statement vs row-level triggers
based on workload characteristics

Executed function must return
appropriate record (`NEW`/`OLD`)
for row-level triggers

Trigger Design Patterns

```
-- Denormalised  
counter update  
CREATE TRIGGER  
decrement_inventory  
AFTER INSERT ON  
order_items  
FOR EACH ROW EXECUTE  
FUNCTION  
inventory.log_sale()  
;
```

```
-- Guard rail  
CREATE TRIGGER  
prevent_negative_bal  
ance  
BEFORE UPDATE ON  
accounts  
FOR EACH ROW  
WHEN (NEW.balance <  
0)  
EXECUTE FUNCTION  
util.raise_if_negati  
ve();
```



Transactions & ACID

```
BEGIN;  
UPDATE accounts  
SET balance =  
balance - 100  
WHERE id = 1;  
UPDATE accounts  
SET balance =  
balance + 100  
WHERE id = 2;  
COMMIT;  --  
ROLLBACK if any  
statement fails
```

```
SET SESSION  
CHARACTERISTICS  
AS TRANSACTION  
ISOLATION LEVEL  
SERIALIZABLE;
```

Atomicity: all statements succeed or none persist (`BEGIN ... COMMIT` / `ROLLBACK`)

Consistency: constraints and triggers maintain valid database states

Isolation: concurrent transactions behave as if executed serially depending on level

Durability: committed changes survive crashes via WAL and checkpoints

Isolation Levels & Concurrency

```
-- Lock a row for transfer
BEGIN;
SET TRANSACTION ISOLATION
LEVEL REPEATABLE READ;
SELECT balance FROM
accounts WHERE id = 1 FOR
UPDATE;
-- perform business logic
COMMIT;

SELECT pid, wait_event,
locktype
  FROM pg_locks JOIN
pg_stat_activity USING
(pid)
 WHERE wait_event IS NOT
NULL;

      BEGIN;
SET TRANSACTION ISOLATION
LEVEL REPEATABLE READ;
SELECT balance FROM
accounts WHERE id = 1 FOR
UPDATE;
-- business logic here
COMMIT;

SELECT pid, locktype, mode
  FROM pg_locks
 WHERE granted = false;
```

PostgreSQL defaults to `READ COMMITTED`; `REPEATABLE READ` and `SERIALIZABLE` increase safety

Phenomena: dirty reads, non-repeatable reads, phantom reads, write skew

Use explicit locking (`SELECT ... FOR UPDATE`) or advisory locks for coordination

Monitor conflicts with
`pg\_stat\_activity`, `pg\_locks`, and
server logs

Transaction Troubleshooting

```
-- Investigate deadlock victims
SELECT log_time, pid, query
      FROM
pg_logical_slot_peek_changes(
  'pg_logical', NULL, NULL)
  WHERE message LIKE
    '%deadlock%';

SET lock_timeout = '3s';
SAVEPOINT before_large_batch;
-- risky statement
ROLLBACK TO SAVEPOINT
  before_large_batch;
-- Watch for long-running statements
SELECT pid, wait_event_type,
wait_event, query
      FROM pg_stat_activity
  WHERE state = 'active';

-- Guard against blocking
SET lock_timeout = '3s';
SAVEPOINT before_batch;
-- risky statement here
ROLLBACK TO SAVEPOINT
  before_batch;
```

01

Detect
deadlocks via
server logs
(`ERROR:
deadlock
detected`) and

02

Keep
transactions
short to reduce
contention;
avoid waiting
on user input

03

Leverage
savepoints for
partial rollbacks
in complex
workflows

04

Instrument
applications
with retry logic
for serialization
failures

Lab & Midterm Preparation

- Lab: create reporting views, enforce constraints, build trigger-based audit history, and experiment with rollbacks
- Deliver notebook outputs plus SQL scripts for reproducibility
- Proposal kickoff: define project dataset, target users, preliminary ERD, and risk register
- Next week: feedback on proposals and deeper dive into performance tuning prerequisites